

OVERCOOKED

Kaitlyn Adams, Malia Chopra, Zihan Sun
CPE 487-A, Dr. Bernard Yett

OBJECTIVE

This project is a single-player **Overcooked-style** cooking game implemented in VHDL on the **Digilent Nexys A7-100T FPGA**. The game is displayed on a monitor using VGA output through a VGA-to-HDMI adapter, and the player controls the game using the Nexys board buttons and switches. Instead of running on software, the game logic, screen rendering, character movement, item interaction, order display, and state transitions are all handled directly through FPGA hardware.

The player controls a chef inside a kitchen layout and completes food orders before time runs out. The kitchen includes ingredient stations, drinks, a cutting board/fryer area, serving counter, plates, and trash can. The main goal is to pick up the correct items, complete the displayed orders, and finish before the timer reaches zero.

GAME FLOW

The game flow is controlled using a finite state machine, shown in Fig. 1. The output screen depends mainly on the current state: **INTRO_SELECT** displays the character selection screen, **GAME_PLAYING** displays the kitchen gameplay screen, **GAME_WIN** displays the win message, and **GAME_OVER** displays the timeout/game-over message. Inputs such as SW2, order completion, and timer expiration are used to trigger transitions between states.

In **INTRO_SELECT**, the player chooses between the male and female chef by toggling **SW0** and **SW1**. When **SW2** is toggled, the FSM enters **GAME_PLAYING**. During gameplay, the timer runs while the player completes the order cards. If all orders are completed, the FSM transitions to **GAME_WIN**. If the timer reaches zero before all orders are completed, the FSM transitions to **GAME_OVER**. From either end state, toggling **SW2** returns the game back to the intro screen for replay.

SYSTEM DESIGN & DEMO

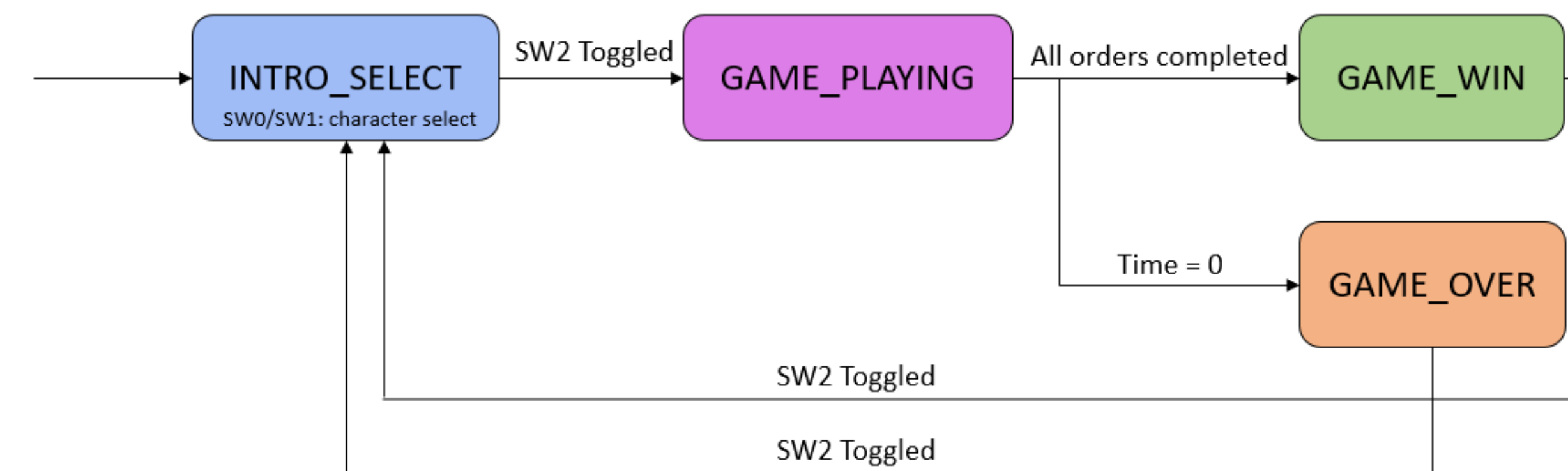


Fig. 1. Finite state machine (Moore FSM) controlling intro selection, gameplay, win state, and game-over state.



Fig. 2. Intro Character Selection Screen (Yellow highlight indicates selection)

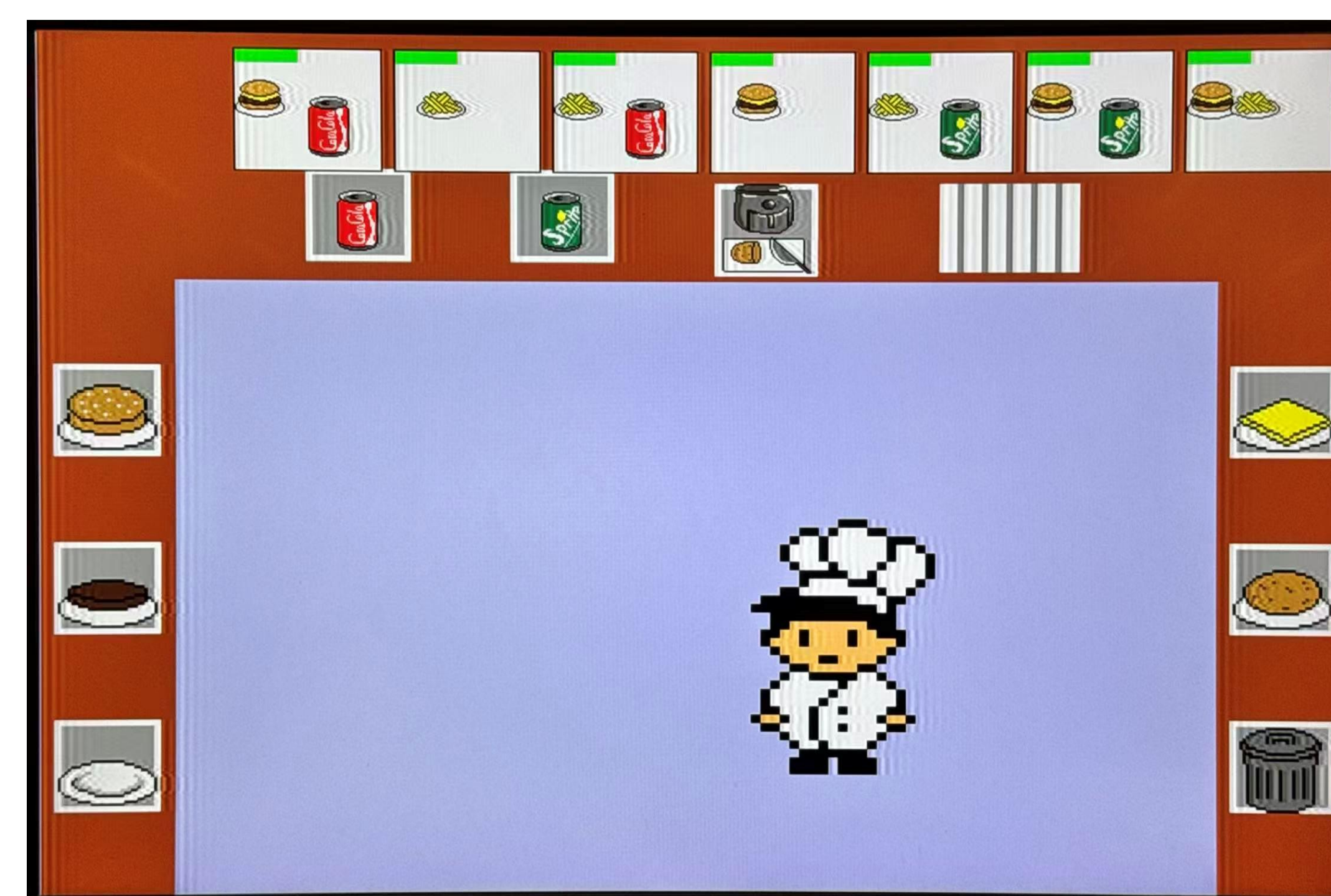


Fig. 3. Gameplay Screen with Orders and Kitchen Layout

GAMEPLAY IMPLEMENTATION

During **GAME_PLAYING**, shown in Fig. 3, the chef is controlled using **BTNU**, **BTND**, **BTNL**, and **BTNR**, while **BTNC** is used for pickup, serving, and trash interactions. Boolean proximity signals such as **near_buns**, **near_cheese**, **near_fryer**, **near_counter**, and **near_trash** determine which station the chef can interact with.

Each item station is displayed inside a grey box with a border to separate it from the kitchen background. When the chef is close enough to interact with an item or station, the border changes to yellow as a visual highlight. This gives the player feedback about which object is currently selectable before pressing BTNC.

The order display shows up to eight shuffled orders using **shuffle_sel**, with each order containing one or two items such as burger, fries, Sprite, or Coke. **current_order**, **item_a_done**, and **item_b_done** track order progress and hide completed items from the display. When all eight orders are completed, the game transitions to **GAME_WIN**, and the score output displays **00001000** in binary on the 7-segment display, representing a completed score of 8. Sprites were drawn as pixel art, converted into VHDL-compatible arrays, and integrated/debugged in Vivado.

RESULTS/FUTURE WORK

Testing was completed through repeated Vivado synthesis, implementation, bitstream generation, and hardware programming. The main debugging challenges included module port mismatches, sprite color issues, sprite positioning, object layering, interaction detection, and state transition behavior. A major part of the project was making sure the game did not only display graphics, but also responded correctly to board inputs and updated the game state based on the player's actions.

Future improvements could include adding a visible score counter, expanding the recipe list, adding more animations, improving the timer warning effect, and organizing sprite memory more efficiently. Another improvement would be making the order shuffling more dynamic with a stronger randomization method instead of predefined shuffled sequences.